1. The dining savages problem

A tribe of savages eats communal dinners from a large pot that can hold *M* servings of stewed missionary. When a savage wants to eat, he helps himself from the pot, unless it is empty. If the pot is empty, the savage wakes up the cook and then waits until the cook has refilled the pot.

Synchronization problems:

- A savage thread might invoke `getServingFromPot` even if the pot is empty;
- Multiple savage threads might decrement `servings` at the same time;
- The cook thread might invoke `putServingsInPot` even if the pot is not empty.

Solution:

```
// Shared variables
servings = 0
mutex = Semaphore(1)
emptyPot = Semaphore(0)
fullPot = Semaphore(0)

// Function for the cook thread
process cook[tid = 1] {
     while True:
          emptyPot.wait()
          putServingsInPot(M)
          fullPot.signal()
}

// Function for the savage thread
process savage[tid = 1 to N] { // Assume there are N savages
     while True:
          mutex.wait()
          if servings == 0:
               emptyPot.signal()
               fullPot.wait()
               servings = M
          servings -= 1
          getServingFromPot()
          mutex.signal()
          eat()
}
```

2. The barbershop problem

A barbershop consists of a waiting room with *n* chairs, and the barber room containing the barber chair. If there are no customers to be served, the barber goes to sleep. If a customer enters the barbershop and all chairs are occupied, then the customer leaves the shop. If the barber is busy, but the chairs are available, then the customer sits in one of the free chairs. If the barber is asleep, the customer wakes the barber.

Additional info:

- Customer threads should invoke a function named `getHaircut`;
- If a customer thread arrives when the shop is full, it can invoke `balk`, which does not return;
- The barber thread should invoke `cutHair`;
- When the barber invokes `cutHair` there should be exactly one thread invoking `getHaircut` concurrently.

Synchronization problems:

- Multiple customer threads might increment `customers` at the same time, meaning that two customers would sit on the same chair in the waiting room;
- A customer thread might invoke `getHaircut` even though the barber chair is occupied;
- The barber thread might invoke `cutHair` even though there is no customer on the barber chair.

Solution:

```
// Shared variables
N = 4 // total number of customers that can be in the shop
customers = 0 // counts the number of customers in the shop
mutex = Semaphore(1)
customer = Semaphore(0)
barber = Semaphore(0)
customerDone = Semaphore(0)
berberDone = Semaphore(0)

// Function for the barber thread
process barber[tid = 1] {
     customer.wait()
     barber.signal()

     cutHair()

     customerDone.wait()
```

```
     barberDone.signal()
}

// Function for the customer thread
process customer[tid = 1 to N] {
     mutex.wait()
     if customers == N:
          mutex.signal()
          balk()
     customers += 1
     mutex.signal()

     customer.signal()
     barber.wait()

     getHaircut()

     customerDone.signal()
     barberDone.wait()

     mutex.wait()
     customers -= 1
     mutex.signal()
}
```

3. The FIFO barbershop

In the previous solution there is no guarantee that customers are served in the order they arrive. Up to *n* customers can pass the turnstile, signal **customer** and wait on **barber**. When the barber signals **barber**, any of the customers might proceed. Modify the solution so that customers are served in the order they pass the turnstile.

Solution:

```
// Shared variables
N = 4 // total number of customers that can be in the shop
customers = 0 // counts the number of customers in the shop
mutex = Semaphore(1)
customer = Semaphore(0)
customerDone = Semaphore(0)
berberDone = Semaphore(0)

// As each thread passes the turnstile, it creates a semaphore and
// puts it in the queue.
// Instead of waiting on barber, each thread waits on its own
// semaphore. When the barber wakes up, he removes a semaphore
// from the queue and signals it.
queue = []

// Function for the barber thread
process barber[tid = 1] {
     customer.wait()
     mutex.wait()
     sem = queue.pop(0)
     mutex.signal()

     sem.signal()

     cutHair()

     customerDone.wait()
     barberDone.signal()
}

// Function for the customer thread
process customer[tid = 1 to N] {
     self.sem = Semaphore(0)
     mutex.wait()
     if customers == N:
          mutex.signal()
```

```
          balk()
     customers += 1
     queue.append(self.sem)
     mutex.signal()

     customer.signal()
     self.sem.wait()

     getHaircut()

     customerDone.signal()
     barberDone.wait()

     mutex.wait()
     customers -= 1
     mutex.signal()
}
```

4. The Santa Claus problem

Santa Claus sleeps in his shop at the North Pole and can only be awakened by either (1) all nine reindeer being back from their vacation in the South Pacific, or (2) some of the elves having difficulty making some toys. To allow Santa to have some sleep, the elves can only wake him up when three of them have problems. When three elves are having their problems solved, any other elves wishing to visit Santa must wait for those elves to return. If Santa wakes up to find three elves waiting at his shop's door, along with the last reindeer having come back from the tropics, Santa has decided that the elves can wait until after Christmas, because it is more important to get his sleigh ready. (It is assumed that the reindeer do not want to leave the tropics, and therefore they stay there until the last possible moment.) The last reindeer to arrive must get Santa while the others wait in a warming hut before being harnessed to the sleigh.

Additional info:

- After the ninth reindeer arrives, Santa must invoke `prepareSleigh`, and then all nine reindeer must invoke `getHitched`;
- After the third elf arrives, Santa must invoke `helpElves`. Concurrently, all three elves should invoke `getHelp`;
- All three elves must invoke `getHelp` before any additional elves enter (increment the elf counter).

Santa should run in a loop so he can help many sets of elves. We can assume that there are exactly 9 reindeer, but there may be any number of elves.

Synchronization problems:

- The variables `elves` and `reindeer` might be incremented by multiple threads at the same time, meaning that Santa Claus might not be awakened if two threads increment `reindeer` past 9 and `elves` past 3;
- Santa Claus might be awakened before he needs to.

Solution:

```
// Shared variables
elves = 0
reindeer = 0
santaSem = Semaphore(0)
reindeerSem = Semaphore(0)
elfTex = Semaphore(1)
mutex = Semaphore(1)

// Function for the Santa thread
process Santa[tid = 1] {
     santaSem.wait()
```

```
     mutex.wait()
     if reindeer == 9:
          prepareSleigh()
          reindeerSem.signal(9)
          reindeer -= 9
     else if elves == 3:
          helpElves()
     mutex.signal()
}

// Function for the reindeer thread
process reindeer[tid = 1 to 9] {
     mutex.wait()
     reindeer += 1
     if reindeer == 9:
           santaSem.signal()
     mutex.signal()

     reindeerSem.wait()
     getHitched()
}

// Function for the elf thread
process elf[tid = 1 to N] { // Assume there are N elves
     elfTex.wait()
     mutex.wait()
     elves += 1
     if elves == 3:
          santaSem.signal()
     else:
          elfTex.signal()
     mutex.signal()

     getHelp()

     mutex.wait()
     elves -= 1
     if elves == 0:
          elfTex.signal()
     mutex.signal()
}
```

5. Building H_2O

There are two kinds of threads, oxygen and hydrogen. In order to assemble these threads into water molecules, we have to create a barrier that makes each thread wait until a complete molecule is ready to proceed. As each thread passes the barrier, it should invoke **bond**. You must guarantee that all the threads from one molecule invoke bond before any of the threads from the next molecule do.

In other words:

- If an oxygen thread arrives at the barrier when no hydrogen threads are present, it has to wait for two hydrogen threads;
- If a hydrogen thread arrives at the barrier when no other threads are present, it has to wait for an oxygen thread and another hydrogen thread.

Solution:

```
// Shared variables
oxygen = 0
hydrogen = 0
barrier = Barrier(3)
oxyQueue = Semaphore(0)
hydroQueue = Semaphore(0)
mutex = Semaphore(1)

// Function for the oxygen thread
process oxygen[tid = 1 to N] { // Assume there are N oxygen threads
     mutex.wait()
     oxygen += 1
     if hydrogen >= 2:
          hydroQueue.signal(2)
          hydrogen -= 2
          oxyQueue.signal()
          oxygen -= 1
     else:
          mutex.signal()

     oxyQueue.signal()
     bond()

     barrier.wait()
     mutex.signal()
}

// Function for the hydrogen thread
process hydrogen[tid = 1 to M] { // Assume there are M hydrogen
```

```
                                        threads
     mutex.wait()
     hydrogen += 1
     if hydrogen >= 2 and oxygen >= 1:
          hydroQueue.signal(2)
          hydrogen -= 2
          oxyQueue.signal()
          oxygen -= 1
     else:
          mutex.signal()

     hydroQueue.wait()
     bond()

     barrier.wait()
}
```

6. River crossing problem

Somewhere near Redmond, Washington there is a rowboat that is used by both Linux hackers and Microsoft employees (serfs) to cross a river. The ferry can hold exactly four people; it won't leave the shore with more or fewer. To guarantee the safety of the passengers, it is not permissible to put one hacker in the boat with three serfs, or to put one serf with three hackers. Any other combination is safe.

As each thread boards the boat it should invoke a function called `board`. You must guarantee that all four threads from each boatload invoke `board` before any of the threads from the next boatload do.

After all four threads have invoked `board`, exactly one of them should call a function named `rowBoat`, indicating that that thread will take the oars. It doesn't matter which thread calls the function, as long as one does.

Don't worry about the direction of travel. Assume we are only interested in traffic going in one of the directions.

Solution:

```
// Shared variables
hackers = 0
serfs = 0
barrier = Barrier(4)
hackerQueue = Semaphore(0)
serfQueue = Semaphore(0)
mutex = Semaphore(1)
local isCaptain = False // local variable that indicates which
                           thread should invoke row.

// Function for the hacker threads
process hacker[tid = 1 to N] { // Assume there are N hacker threads
     mutex.wait()
     hackers += 1
     if hackers == 4:
          hackerQueue.signal(4)
          hackers = 0
          isCaptain = True
     else if hackers == 2 and serfs >= 2:
          hackerQueue.signal(2)
          serfQueue.signal(2)
          serfs -= 2
          hackers = 0
          isCaptain = True
     else:
          mutex.signal()
```

```
     hackerQueue.wait()

     board()
     barrier.wait()

     if isCaptain:
          rowBoat()
          mutex.signal()
}

// Function for the serf threads
process serf[tid = 1 to M] { // Assume there are M serf threads
     mutex.wait()
     serfs += 1
     if serfs == 4:
          serfQueue.signal(4)
          serfs = 0
          isCaptain = True
     else if serfs == 2 and hackers >= 2:
          serfQueue.signal(2)
          hackerQueue.signal(2)
          hackers -= 2
          serfs = 0
          isCaptain = True
     else:
          mutex.signal()

     serfQueue.wait()

     board()
     barrier.wait()

     if isCaptain:
          rowBoat()
          mutex.signal()
}
```

7. The roller coaster problem

Suppose there are *n* passenger threads and a car thread. The passengers repeatedly wait to take rides in the car, which can hold *C* passengers, where $C < n$. The car can go around the tracks only when it is full.

Here are some additional details:

- Passengers should invoke `board` and `unboard`;
- The car should invoke `load`, `run` and `unload`;
- Passengers cannot board until the car has invoked `load`;
- The car cannot depart until C passengers have boarded;
- Passengers cannot unboard until the car has invoked `unload`.

Solution:

```
// Shared variables
boarders = 0
unboarders = 0
boardQueue = Semaphore(0)
unboardQueue = Semaphore(0)
allAbroad = Semaphore(0)
allAshore = Semaphore(0)
mutex = Semaphore(1)
mutex2 = Semaphore(1)

// Function for the car thread
process car[tid = 1] {
     load()
     boardQueue.signal(C)
     allAbroad.wait()

     run()

     unload()
     unboardQueue.signal(C)
     allAshore.wait()
}

// Function for the passenger threads
process passenger[tid = 1 to N] { // Assume there are N passenger
                                  threads
     boardQueue.wait()
     board()

     mutex.wait()
```

```
     boarders += 1
     if boarders == C:
          allAbroad.signal()
          boarders = 0
     mutex.signal()

     unboardQueue.wait()
     unboard()

     mutex2.wait()
     unboarders += 1
     if unboarders == C:
          allAshore.signal()
          unboarders = 0
     mutex2.signal()
}
```

8. The sushi bar problem

Imagine a sushi bar with 5 seats. If you arrive while there is an empty seat, you can take a seat immediately. But if you arrive when all 5 seats are full, that means that all of them are dining together, and you will have to wait for the entire party to leave before you sit down.

Solution:

```
// Shared variables
eating = 0
waiting = 0
mutex = Semaphore(1)
block = Semaphore(0)
must_wait = False

// Function for the thread
process sushi_bar[tid = 1 to N] { // Assume there are N threads
     mutex.wait()
     if must_wait:
          waiting += 1
          mutex.signal()
          block.wait()
     else:
          eating += 1
          must_wait = (eating == 5)
          mutex.signal()

     eat_sushi()

     mutex.wait()
     eating -= 1
     if eating == 0
          n = min(5, waiting)
          waiting -= n
          eating += n
          must_wait = (eating == 5)
          block.signal(n)
     mutex.signal()
}
```